Predictive Maintenance Project

1. Introduction

Project title:

Problem statement

Machine failures in manufacturing lead to unexpected downtime and financial loss. This project develops a predictive model using sensor data (temperature, torque, speed) to identify machines at risk of failure in advance. Preventing them is **high priority for any factory**.

The model classifies:

- **0 → Healthy machine**
- **1 → High risk of failure**

It helps answer key business questions like:

- Which machines need maintenance?
- When should maintenance be scheduled?
- Which machines are likely to fail soon?

Objective of the project

To build a machine learning model that predicts potential machine failures using industrial sensor data (temperature, torque, speed) in order to enable proactive maintenance and reduce unexpected downtime, maintenance costs, and production losses.

Business Impact

The solution enables predictive maintenance by identifying high-risk machines early. This helps reduce downtime, lower maintenance costs, improve machine lifespan, and increase production efficiency by shifting from reactive to proactive maintenance.

2. Dataset Description

Source of dataset

Kaggle dataset: https://www.kaggle.com/datasets/stephanmatzka/predictive-maintenance-dataset-ai4i-2020

Number of rows & columns

Rows: 10000

Columns: 14

Feature descriptions & Target variable explanation

1. UID: unique identifier ranging from 1 to 10000
2. product ID: consisting of a letter L, M, or H for low (50% of all products), medium (30%) and high (20%) as product quality variants and a variant-specific serial number
3. type: just the product type L, M or H from column 2
4. air temperature [K]: generated using a random walk process later normalized to a standard deviation of 2 K around 300 K
5. process temperature [K]: generated using a random walk process normalized to a standard deviation of 1 K, added to the air temperature plus 10 K.
6. rotational speed [rpm]: calculated from a power of 2860 W, overlaid with a normally distributed noise
7. torque [Nm]: torque values are normally distributed around 40 Nm with a SD = 10 Nm and no negative values.
8. tool wear [min]: The quality variants H/M/L add 5/3/2 minutes of tool wear to the used tool in the process.
9. a 'machine failure' label that indicates, whether the machine has failed in this particular datapoint for any of the following failure modes are true.

The machine failure consists of five independent failure modes

1. tool wear failure (TWF): the tool will be replaced of fail at a randomly selected tool wear time between 200 - 240 mins (120 times in our dataset). At this point in time, the tool is replaced 69 times, and fails 51 times (randomly assigned).
2. heat dissipation failure (HDF): heat dissipation causes a process failure, if the difference between air- and process temperature is below 8.6 K and the tools rotational speed is below 1380 rpm. This is the case for 115 data points.
3. power failure (PWF): the product of torque and rotational speed (in rad/s) equals the power required for the process. If this power is below 3500 W or above 9000 W, the process fails, which is the case 95 times in our dataset.
4. overstrain failure (OSF): if the product of tool wear and torque exceeds 11,000 minNm for the L product variant (12,000 M, 13,000 H), the process fails due to overstrain. This is true for 98 datapoints.
5. random failures (RNF): each process has a chance of 0,1 % to fail regardless of its process parameters. This is the case for only 5 datapoints, less than could be expected for 10,000 datapoints in our dataset.
   If at least one of the above failure modes is true, the process fails and the 'machine failure' label is set to 1. It is therefore not transparent to the machine learning method, which of the failure modes has caused the process to fail.

3. Exploratory Data Analysis (EDA)

1. Descriptive Analysis

I started with a descriptive analysis to understand the overall dataset. The dataset is highly imbalanced: there are 9,661 non-failure instances and 339 failure instances. Among failure types, the most common is HDF (Heat dissipation Failure), followed by Overstrain, Power Failure, Tool Wear, and other random types.

2. Product Type and Failure Rate

I analyzed the effect of product type on machine failure. It was observed that low-quality products have the highest failure rates, indicating that product quality significantly influences failures.

3. Statistical and Hypothesis Testing (Inferential Analysis)

I compared input features across machine failure and non-failure cases. Significant differences were observed in all input features between failed and non-failed machines. I also analyzed the relationship between product quality and machine failure, finding a statistically significant difference. However, when checking input features across product quality levels, no significant differences were found.

4. Root Cause and Risk Analysis

To identify which input parameters have the strongest impact on failure, I created a correlation heatmap. Torque emerged as the most influential parameter. Using boxplots, I observed that failures often occur at extreme values of rotational speed and torque. For risky combinations, scatterplots revealed that high torque + low speed or low torque + high speed conditions are associated with failures. These combinations represent the primary mechanical risk scenarios.

5. Recommendations and Actionable Insights

I calculated critical thresholds for input features using percentiles (25th, 50th, 75th) and assessed their correlation with machine failure. Identified a safe operating range where machine failure probability is extremely low (~0.04%), ensuring operational reliability. Grouping by product type, high-quality products were found to be the most reliable.

6. Summary

Failures are rare but mostly occur in low-quality products and at extreme machine parameters. Torque is the most critical parameter impacting failure. High torque + low speed is the riskiest condition. Safe operating thresholds have been identified to minimize risk.

Data types (numerical, categorical)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 14 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   UDI                      10000 non-null  int64
 1   Product ID               10000 non-null  object
 2   Type                     10000 non-null  object
 3   Air temperature [K]      10000 non-null  float64
 4   Process temperature [K]  10000 non-null  float64
 5   Rotational speed [rpm]   10000 non-null  int64
 6   Torque [Nm]              10000 non-null  float64
 7   Tool wear [min]          10000 non-null  int64
 8   Machine failure          10000 non-null  int64
 9   TWF                      10000 non-null  int64
 10  HDF                      10000 non-null  int64
 11  PWF                      10000 non-null  int64
 12  OSF                      10000 non-null  int64
 13  RNF                      10000 non-null  int64
dtypes: float64(3), int64(9), object(2)
memory usage: 1.1+ MB
```

NUMERICAL COLUMNS:

['UDI', 'Air temperature [K]', 'Process temperature [K]', 'Rotational speed [rpm]', 'Torque [Nm]', 'Tool wear [min]', 'Machine failure', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF'] and

CATEGORICAL COLUMNS:

['Product ID', 'Type']

Missing values analysis

```
UDI                        0
Product ID                 0
Type                       0
Air temperature [K]        0
Process temperature [K]    0
Rotational speed [rpm]     0
Torque [Nm]                0
Tool wear [min]            0
Machine failure            0
TWF                        0
HDF                        0
PWF                        0
OSF                        0
RNF                        0
dtype: int64
```

Statistical summary

	UDI	Air temperature [K]	Process temperature [K]	Rotational speed [rpm]	Torque [Nm]	Tool wear [min]	Machine failure	TWF	HDF	PWF	OSF	RNF
Mean	5000.5	300.00493	310.00556	1538.7761	39.98691	107.951	0.0339	0.0046	0.0115	0.0095	0.0098	0.0019
Median	5000.5	300.1	310.1	1503.0	40.1	108.0	0.0	0.0	0.0	0.0	0.0	0.0
Mode	1	300.7	310.6	1452	40.2	0	0	0	0	0	0	0
Q1:25%	2500.75	298.3	308.8	1423.0	33.2	53.0	0.0	0.0	0.0	0.0	0.0	0.0
Q2:50%	5000.5	300.1	310.1	1503.0	40.1	108.0	0.0	0.0	0.0	0.0	0.0	0.0
Q3:75%	7500.25	301.5	311.1	1612.0	46.8	162.0	0.0	0.0	0.0	0.0	0.0	0.0
99%	9900.01	304.1	313.2	2188.01	62.8	222.0	1.0	0.0	1.0	0.0	0.0	0.0
Q4:100%	10000.0	304.5	313.8	2886.0	76.6	253.0	1.0	1.0	1.0	1.0	1.0	1.0
IQR	4999.5	3.2	2.3	189.0	13.6	109.0	0.0	0.0	0.0	0.0	0.0	0.0
1.5_rule	7499.25	4.8	3.45	283.5	20.4	163.5	0.0	0.0	0.0	0.0	0.0	0.0
lower_bound	-4998.5	293.5	305.35	1139.5	12.8	-110.5	0.0	0.0	0.0	0.0	0.0	0.0
upper_bound	14999.5	306.3	314.55	1895.5	67.2	325.5	0.0	0.0	0.0	0.0	0.0	0.0
min	1	295.3	305.7	1168	3.8	0	0	0	0	0	0	0
max	10000	304.5	313.8	2886	76.6	253	1	1	1	1	1	1
skew	0.0	0.114274	0.015027	1.993171	-0.009517	0.027292	5.151852	14.644462	9.164789	10.114516	9.953916	22.87957
kurtosis	-1.2	-0.835962	-0.499734	7.392945	-0.013241	-1.166737	24.546486	212.502762	82.009755	100.323504	97.099856	521.579046
Var	8334166.666667	4.001035	2.201467	32142.787047	99.37964	4051.850384	0.032754	0.004579	0.011369	0.009411	0.009705	0.001897
Std_dev	2886.89568	2.000259	1.483734	179.284096	9.968934	63.654147	0.180981	0.067671	0.106625	0.097009	0.098514	0.04355

Outlier detection

find_outlier

```
least_outlier, greater_outlier = Univariate.find_outlier(descriptive_analysis,quan)
print(f"least_outlier : {least_outlier} and\ngreater_outlier: {greater_outlier}")
```

```
least_outlier : ['Torque [Nm]'] and
greater_outlier: ['Rotational speed [rpm]', 'Torque [Nm]', 'Machine failure', 'TWF', 'HDF', 'PWF', 'OSF', 'RNF']
```

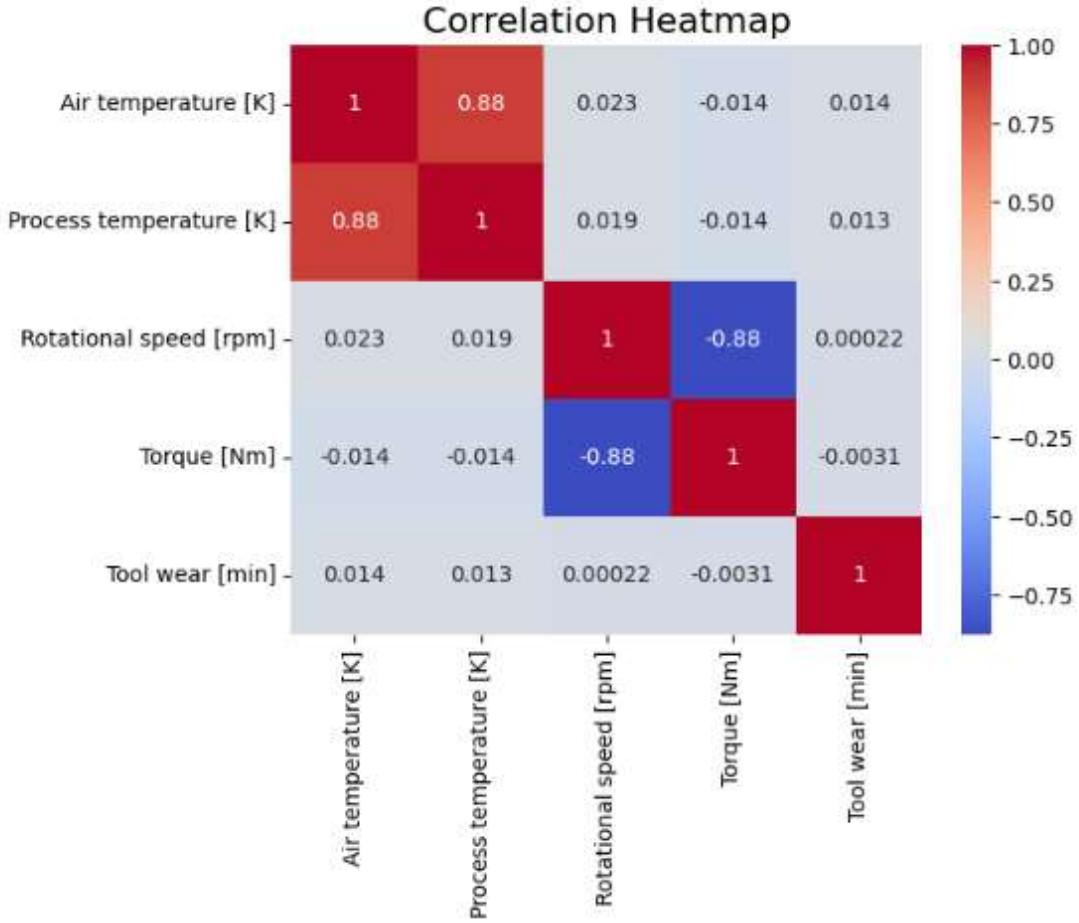
KEEPING OUTLIERS EXACTLY AS HOW THEY ARE

In a "normal" data science project (like predicting house prices), an outlier is often an error. In Predictive Maintenance, an outlier is a signal.

I performed an outlier analysis on the sensor telemetry. While several features (Torque, RPM) contained statistical outliers, I chose to retain them because, in a predictive maintenance context, these anomalies represent critical failure modes. To ensure model stability

THESE OUTLIERS ==> HANDLED USING RobustScaler (without capping) - STANDARDIZATION

Correlation analysis



4. Data Preprocessing

I started my predictive maintenance machine learning project by first understanding the dataset. I loaded the dataset and examined its shape, which contains 10,000 rows and 14 columns. Using the .info() function, I identified the data types and observed that UID and Product ID are unique identifier columns, Type is a categorical feature, and the remaining variables such as air temperature, process temperature, rotational speed, torque, and tool wear are numerical features. The target variable is Machine failure, which represents whether a failure has occurred. I then used descriptive statistics to understand the distribution, central tendency, and variability of the features. After that, I checked for missing values and found that the dataset has no null values, so no imputation was required. I removed irrelevant columns like UID and Product ID as they do not contribute to the prediction task. To handle the categorical variable Type, I applied encoding to convert it into a numerical format. Finally, I analyzed the target variable distribution and found that the dataset is imbalanced, which is an important consideration for model building. These steps formed the initial preprocessing phase before proceeding to exploratory data analysis.

Encoding (Label Encoding / One-Hot Encoding)

Encoding - Preprocessing

convert categorical data into numerical data

```
# Encoding - Preprocessing
## convert categorical data into numerical data

from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
dataset['Type'] = label_encoder.fit_transform(dataset['Type'])
'''  # LabelEncoder assigns values based on alphabetical order
L  -> 1
M  -> 2
H  -> 0
'''
```

```
'  # LabelEncoder assigns values based on alphabetical order\nL  -> 1\nM  -> 2\nH  -> 0\n'
```

save the labelEncoder preprocessing

```
import pickle
with open('lable_encoder.sav','wb') as file:
    pickle.dump(label_encoder, file)
```

Feature scaling (StandardScaler / RobustScaler)

```
x = dataset.drop(['Machine failure'],axis=1)
x
# using Robust Scaler --> Keeps the exact extreme values but scales the "normal" data around them.
from sklearn.preprocessing import RobustScaler
transformer = RobustScaler()
x = transformer.fit_transform(x)
pd.DataFrame(x)
```

	0	1	2	3	4	5	6
0	1.0	-0.62500	-0.652174	0.253968	0.198529	-0.990826	0.411765
1	0.0	-0.59375	-0.608696	-0.502646	0.455882	-0.963303	0.411765
2	0.0	-0.62500	-0.695652	-0.026455	0.683824	-0.944954	0.352941
3	0.0	-0.59375	-0.652174	-0.370370	-0.044118	-0.926606	0.352941
4	0.0	-0.59375	-0.608696	-0.502646	-0.007353	-0.908257	0.411765
...	...	...	...	...	...	...	...
9995	1.0	-0.40625	-0.739130	0.534392	-0.779412	-0.862385	-0.117647
9996	-1.0	-0.37500	-0.739130	0.682540	-0.610294	-0.834862	-0.176471
9997	1.0	-0.34375	-0.652174	0.751323	-0.492647	-0.788991	-0.117647
9998	-1.0	-0.34375	-0.608696	-0.502646	0.617647	-0.761468	-0.058824
9999	1.0	-0.34375	-0.608696	-0.015873	0.007353	-0.715596	-0.058824

10000 rows × 7 columns

Train-test split

train test split

```
[6]: from sklearn.model_selection import train_test_split
     x_train, x_test, y_train, y_test = train_test_split(x,y,test_size=0.30,random_state=42)
```

```
[7]: for i in y_train, y_test:
         counts = pd.DataFrame(i).value_counts()
         print(counts)
     # y_train -> is imbalanced - handled using SMOTE
```

```
Machine failure
0                  6754
1                   246
Name: count, dtype: int64
Machine failure
0                  2907
1                    93
Name: count, dtype: int64
```

5. Feature Engineering

Created features

```
# adding new feature - Temp_diff -> thermal stress
dataset['Temp_diff [K]'] = dataset['Process temperature [K]'] - dataset['Air temperature [K]']
```

```
dataset['power'] = dataset['Torque [Nm]'] * dataset['Rotational speed [rpm]']
dataset['wear_rate'] = dataset['Tool wear [min]'] / dataset['Rotational speed [rpm]']
```

Why these features are useful

“I calculated temp_diff = process_temp – air_temp because it normalizes machine temperature for ambient conditions. High deviations indicate abnormal heating, which often precedes mechanical or tool failure. This makes the model more sensitive to early failure signals.”

"I created features like Power (Torque × Rotational Speed) and Wear Rate (Tool Wear ÷ Rotational Speed) because they capture **combined mechanical stress and degradation rate**. These features give the model a clearer signal of when the machine is under abnormal load or wearing too fast, which often precedes failure."

6. Handling Imbalanced Data

Problem explanation

Class imbalance occurs when one class in a dataset significantly outnumbers the other(s), leading to biased models that favor the majority class. This can result in poor performance for the minority class, which is often the focus of the analysis.

Technique used

SMOTE (Synthetic Minority Oversampling Technique)

handling imbalanced data - for training set

```
from imblearn.over_sampling import SMOTE    # need to be applied only in training data - to avoid data leakage
smote = SMOTE(random_state=42)
x_train_resampled, y_train_resampled = smote.fit_resample(x_train_scaled, y_train)
```

```
y_train_resampled.value_counts()
```

```
Machine failure
0    6754
1    6754
Name: count, dtype: int64
```

Before vs after distribution

BEFORE

```
Machine failure
0                  6754
1                   246
Name: count, dtype: int64
```

AFTER

```
Machine failure
0    6754
1    6754
Name: count, dtype: int64
```

7. Model Building and Final Model Selection

- Built a classification model to predict **machine failure (1) or non-failure (0)** using a Kaggle dataset.
- Split data into **train and test sets** and separated input features and target variable.
- Applied **RobustScaler** to handle outliers without losing important failure-related patterns.
- Used **SMOTE** to balance the dataset and avoid model bias toward non-failure cases.
- Trained multiple models: Logistic Regression, KNN, SVC, Naive Bayes, Decision Tree, Random Forest, and Gradient Boosting.
- Evaluated models using **confusion matrix and classification report**, focusing on reducing **false negatives** (critical for failure prediction).
- Selected **Decision Tree and Random Forest** as best models.
- Performed **GridSearchCV with F1-weighted score** for tuning.
- Final model: **Random Forest Classifier** with **~97% accuracy** and low false negatives.
- Saved the final model for deployment.

8. Model Evaluation

- For evaluation, I used **Confusion Matrix** and **Classification Report** since it is a classification problem.
- From the **confusion matrix**, I analyzed:
- True Positives (TP)
- True Negatives (TN)
- False Positives (FP) → false alarms (machine is fine but predicted as failure)
- False Negatives (FN) → most critical error (machine fails but not detected)
- In this problem, **false negatives are most important to minimize**, because missing a machine failure can lead to **downtime and high financial loss**.
- **False positives** were also considered important to reduce, as they can cause unnecessary maintenance actions and operational panic.
- From the **classification report**, I evaluated:
- **Precision** → correctness of positive predictions
- **Recall** → ability to correctly detect actual failures
- **F1-score** → balance between precision and recall
- **Accuracy** → overall performance of the model
- Final model selection was based on achieving a **balanced trade-off**, with priority given to **high recall and low false negatives** as per business requirement.

9. Hyperparameter Tuning

- For hyperparameter tuning, I used **GridSearchCV**.
- It performs **exhaustive search over all possible parameter combinations** of a given algorithm.
- The model is evaluated using **5-fold cross-validation**, ensuring reliable and unbiased performance estimation.
- The evaluation metric used was **F1-weighted score**, which balances precision and recall for imbalanced datasets.
- GridSearchCV selects the **best combination of hyperparameters** based on the highest cross-validation score.
- This helped in improving model performance and ensuring better generalization.

10. Model Deployment (Basic)

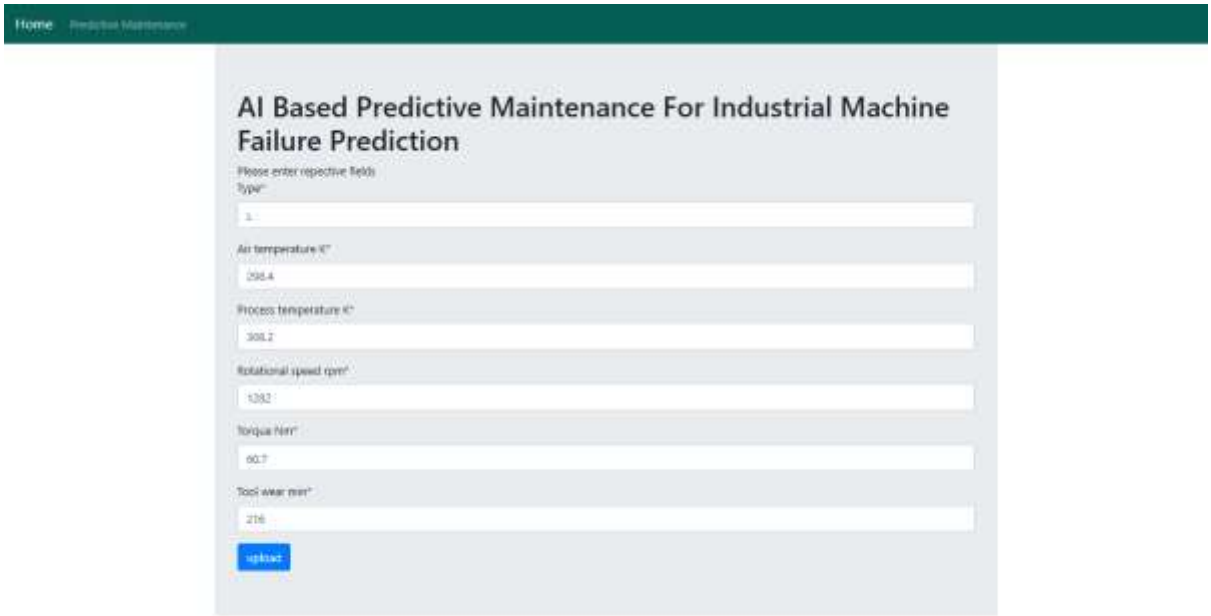
- For deployment, I saved the trained model, label encoder, and RobustScaler to ensure consistency in preprocessing and prediction.
- These saved artifacts are loaded during inference to apply the same transformations used during training.
- Input data from the user is first preprocessed using:

- Label Encoder (for categorical variables)
- RobustScaler (for feature scaling and outlier handling)
- After preprocessing, the data is passed to the trained model to generate predictions.
- The model is deployed using a web-based application built with Django (backend) and HTML forms (frontend).
- User inputs are collected through the web interface, processed, and predictions are displayed in real time.
- This ensures:
- Consistency between training and deployment pipeline
- Accurate and error-free predictions
- End-to-end working machine learning application

Screenshots of the deployed system are included for reference.

MACHINE NON FAILURE:

MACHINE FAILURE:



11. Conclusion

The project focuses on predicting whether a machine will fail or not in advance using machine learning. This helps industries shift from **reactive maintenance to predictive maintenance**, which reduces downtime, minimizes financial loss, improves machine lifespan, and increases production efficiency. It also enables better planning of maintenance schedules and production loads, solving a critical problem in manufacturing systems.

12. Future Improvements

- Extend the model to predict **type of failure** (e.g., heat dissipation failure, tool wear, overstrain, power failure, random failure).
- Further reduce prediction errors, especially **false negatives**, by improving model tuning and feature engineering.
- Improve model robustness beyond current performance (Random Forest baseline).
- Integrate the system with **IoT-enabled machines** for real-time data collection and prediction.
- Develop a full **industrial-scale deployment system** where models communicate directly with machine sensors.
- Conduct deeper **R&D per machine type**, since each machine has unique hardware behavior and failure patterns.
- **Final Insight**

With better integration of IoT systems and advanced modeling, this solution can evolve into a real-time intelligent maintenance system that optimizes machine performance and operational efficiency in manufacturing industries.

14. References

https://www.youtube.com/watch?v=YwTWpUl3QS8

https://www.youtube.com/watch?v=InMlOMcUzM4

https://www.semanticscholar.org/reader/402303ecf9fe73d152543a4e78d4014f1c7e2df2